My Exam

Task 1:

1.1

1.1

To calculate MLE of w, we use a following equation

$$W_{MLE} = (\Phi^T \Phi)^{-1} \Phi^T y \quad , \text{ where } \Phi = [1, x^2, \sin(x), x]$$

- I obtain:

$$W_{MLE} = \begin{bmatrix} -0.73 \\ 0.11 \\ 2.35 \\ 1.07 \end{bmatrix}$$

To calculate σ^2_{MLE} we use $\frac{1}{N} \sum_{i=1}^{N} (y_i - w_{MLE}^T \phi(x_i))^2$

I obtain $\sigma^2_{MLE} = 0.26$, to obtain B, we set $B = \frac{1}{\sigma^2} = 4.84$

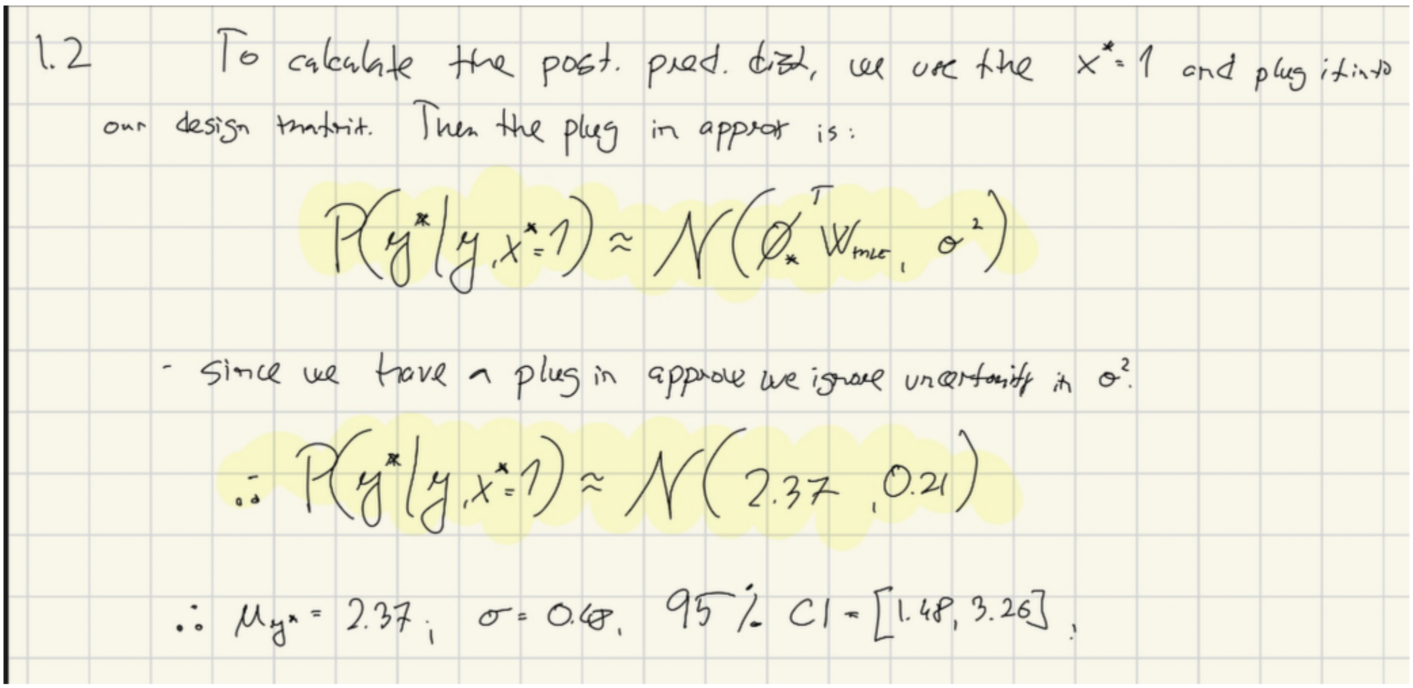
Figure 1: MLE and Beta OLS estimate

CODE

```
x = jnp.array([
    2.29, -1.8, -0.06, 3.72, 2.6, -5.93, -0.15
])
y = jnp.array([
    3.17, -4.53, -0.78, 3.15, 4.76, -1.96, -1.32
]).reshape(-1, 1)

def design_matrix(x):
    return jnp.column_stack((jnp.ones(len(x)), x**2, jnp.sin(x), x))

PHI = design_matrix(x)


model = BR.BayesianLinearRegression(Phi=PHI, y=y, beta=1, alpha=1)

```

```
w_MLE = model.get_w_mle()
print(f"The MLE of W is \n{w_MLE}")
The MLE of W is
[[-0.73483423]
 [ 0.11247988]
 [ 2.35754774]
 [ 1.01247728]]

sigma = model.get_sigma2_mle()
print(f"The estimate of sigma is {sigma}")
print(f"We obtain beta by 1/sigma² = {1/sigma}")
The estimate of sigma is 0.20655746802181824
We obtain beta by 1/sigma² = 4.841267709063765
```

1.2



1.2 To calculate the post. pred. dist, we use the $x^* = 1$ and plug it into our design matrix. Then the plug in approx is:

$$P(y^* | y, x^* = 1) \approx \mathcal{N}(\phi_*^T W_{mle}, \sigma^2)$$

- Since we have a plug in approx we ignore uncertainty in σ^2.

$$\therefore P(y^* | y, x^* = 1) \approx \mathcal{N}(2.37, 0.21)$$

$\therefore \mu_{y^*} = 2.37$, $\sigma = 0.48$, 95% CI = [1.48, 3.26].

Figure 2: Post. pred. dist. mean, std and CI

```
x_star = jnp.array([1])

Phi_star = design_matrix(x_star)
print(Phi_star)
print(f"Phi_star_shape {Phi_star.shape}")


mean_plug_in = Phi_star @ w_MLE
print(f"The mean of post. pred. dist p(y*|y, x*=1) = {mean_plug_in.item():.2f}")

mean_plug_in = Phi_star @ w_MLE

print(f"The mean of post. pred. dist p(y*|y, x*=1) = {mean_plug_in.item():.2f}")

print(f"Variance is just {sigma:.2f}")
print(f"STD is {jnp.sqrt(sigma):.2f}")
```

```python
print(f"Credibility intervals are {GCI.gaussian_credibility_interval(mean_plug_in, sigma)}")
```

Task 1:

Task 1:

Task 1:

Task 1: